# Claude Certified Architect – Foundations

Scenario-Based Mock Exam (Set 2) · 60 questions across 6 production scenarios · 120 minutes

| Exam detail | Value (as published by prep sources, Oct 2026) |
|---|---|
| Exam | Claude Certified Architect – Foundations (CCA-F) |
| Format | 60 scenario-based multiple-choice questions, 120 minutes (2 min per question). Each question has one correct answer and three distractors. |
| Scoring | Scaled 100–1,000; passing score 720 |
| Delivery | Pearson VUE; closed book |

| # | Domain | Weight | Questions here |
|---|---|---|---|
| 1 | Agentic Architecture & Orchestration | 27% | 16 |
| 2 | Tool Design & MCP Integration | 18% | 11 |
| 3 | Claude Code Configuration & Workflows | 20% | 12 |
| 4 | Prompt Engineering & Structured Output | 20% | 12 |
| 5 | Context Management & Reliability | 15% | 9 |
| | **Total** | **100%** | **60** |

**Important:** These are original practice questions written to match the published CCA-F domain blueprint and the exam's scenario-based style. They are **not** actual exam questions, and Anthropic has not reviewed or endorsed them. Domain weights and format come from third-party prep sources; check them against Anthropic's official exam guide. This set does not share questions with Set 1 (the domain question bank).

# Exam Instructions

The real CCA-F presents questions inside production scenarios. Each scenario below starts with a short context, followed by 10 questions. Read the scenario carefully: details such as "Agent SDK", "CI pipeline", or "compliance requirement" often decide the answer.

Set a 120-minute timer and work through all six scenarios without notes. Mark your answers on the answer sheet on the next page.

Each question has exactly one best answer. Several options will sound reasonable. Choose the one that addresses the root cause with the most reliable mechanism.

Target: 44/60 or more (about 73%) as a rough stand-in for the 720/1,000 pass mark. Aim for 48+ for a safety margin.

## Answer Sheet

| Q | Ans | Flag | Q | Ans | Flag | Q | Ans | Flag | Q | Ans | Flag |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | | | 16 | | | 31 | | | 46 | | |
| 2 | | | 17 | | | 32 | | | 47 | | |
| 3 | | | 18 | | | 33 | | | 48 | | |
| 4 | | | 19 | | | 34 | | | 49 | | |
| 5 | | | 20 | | | 35 | | | 50 | | |
| 6 | | | 21 | | | 36 | | | 51 | | |
| 7 | | | 22 | | | 37 | | | 52 | | |
| 8 | | | 23 | | | 38 | | | 53 | | |
| 9 | | | 24 | | | 39 | | | 54 | | |
| 10 | | | 25 | | | 40 | | | 55 | | |
| 11 | | | 26 | | | 41 | | | 56 | | |
| 12 | | | 27 | | | 42 | | | 57 | | |
| 13 | | | 28 | | | 43 | | | 58 | | |
| 14 | | | 29 | | | 44 | | | 59 | | |
| 15 | | | 30 | | | 45 | | | 60 | | |

# Scenario 1: Customer Support Resolution Agent

**Scenario context.** You are building a customer support agent with the Claude Agent SDK for an online retailer. The agent uses MCP tools get_customer, lookup_order, process_refund, and escalate_to_human. The business goal is at least 80% first-contact resolution while staying compliant with refund policy and knowing when to hand off to a human.

**Question 1.** A customer writes: "My order ORD-11902 arrived damaged, I was double-charged for ORD-11877, and I want to update my shipping address." How should the agent handle this message?

**A.** Resolve the first issue mentioned, then ask the customer to start a new conversation for each of the remaining issues afterwards.

**B.** Ask the customer which of the three issues is most important to them, and handle only that one in the current conversation.

**C.** Break it into three distinct issues, investigate each (in parallel where they are independent), then write one unified reply covering all three.

**D.** Escalate to a human immediately, since a message with several issues is too complex for the agent to resolve reliably on its own.

**Question 2.** In one response, Claude returns a text block ("Let me check that refund for you.") together with a tool_use block for process_refund. The developer's loop treats any text block as the final answer and exits, so the refund never runs. What is the fix?

**A.** Check whether the text ends with a question mark, and treat only responses without a question as final answers to the customer.

**B.** Base loop control on stop_reason: when it is "tool_use", execute the requested tools and continue, whatever text is also present.

**C.** Instruct Claude never to include text in responses that contain tool calls, so text reliably marks the end of the agent's work.

**D.** Strip all text blocks from responses before processing, and execute tools only when the response contains no text at all.

**Question 3.** A customer asks the agent to match a competitor's lower price. The company's refund and pricing policy says nothing about price matching. What should the agent do?

**A.** Escalate to a human, because the request falls into a policy gap that the agent should not resolve by improvising its own rule.

**B.** Search the web for the competitor's price to verify it, then apply the difference as a refund if the claim checks out.

**C.** Approve a partial price match as a goodwill gesture, since customer satisfaction is a priority and the policy does not prohibit it.

**D.** Decline the request firmly, since anything that is not explicitly allowed by the written policy should be treated as forbidden.

**Question 4.** get_customer returns three accounts that all match the name "J. Patel". What should the agent do next?

**A.** Proceed with all three accounts, and look up orders across each of them to find the one that the customer is referring to.

**B.** Escalate to a human immediately, since identity ambiguity always requires manual verification by support staff.

**C.** Select the account with the most recent activity, since it is statistically the most likely to belong to the current customer.

**D.** Ask the customer for an additional identifier, such as an email address or order number, to determine which account is theirs.

**Question 5.** lookup_order sometimes fails because of brief network timeouts with the order service. Currently it returns "lookup failed", and the agent tells customers their order cannot be found. Which tool response is best?

**A.** A structured error marked transient and retryable, with a description, so the agent retries before telling the customer.

**B.** The raw exception text from the order service's client library, so the agent has the full technical detail of the failure.

**C.** An empty order object, so the agent can continue the conversation without being interrupted by a temporary service problem.

**D.** A permission error, so the agent asks the customer to verify their identity again before the lookup is attempted a second time.

**Question 6.** Product managers want to add 12 more tools to the support agent (inventory forecasting, marketing campaigns, warehouse scheduling, and others), because "the agent might need them someday". What should the architect advise?

**A.** Add the tools with detailed descriptions, since modern models handle large tool sets well when every tool is clearly documented.

**B.** Add the tools but forbid their use in the system prompt until a real customer need appears in the support conversation logs.

**C.** Keep the agent's tool set focused on support tasks. Extra tools degrade selection, and out-of-scope needs belong to other agents.

**D.** Add all 12 tools, but put the support tools first in the definitions, so the agent prefers them when choosing which tool to call.

**Question 7.** To answer policy questions, the agent repeatedly calls a search tool to find out which policy documents exist before reading the right one. Each conversation spends 3 to 5 calls on this. How should the MCP integration be improved?

**A.** Paste every policy document in full into the system prompt, so the agent never needs to search for policy information.

**B.** Expose the policy documents as MCP resources, so the agent can see the available catalogue without exploratory searching.

**C.** Increase the per-conversation tool budget, so exploratory policy searches do not crowd out other customer actions.

**D.** Cache search results across conversations, so the second search for a given policy term returns instantly.

**Question 8.** Twenty-five turns into a conversation, older turns have been summarised. The agent now refers to a refund of "about $250" when the customer said $247.83, and it has lost the second order number. What is the best fix?

**A.** Disable summarisation for support conversations, and keep the complete transcript in context however long it becomes.

**B.** Tune the summariser prompt so that it preserves all numbers, dates, and order identifiers exactly as the customer stated them.

**C.** Ask the customer to restate key details whenever the agent is about to take an action involving money or orders.

**D.** Maintain a structured case-facts block (amounts, order IDs, dates, promised actions) that is included verbatim in every prompt.

**Question 9.** A customer is visibly frustrated ("This is ridiculous, third time contacting you!") but has not asked for a human. Their issue is straightforward, and the agent can resolve it now. What is the best behaviour?

**A.** Ignore the emotional content entirely and proceed with the resolution steps, to keep the interaction efficient and short.

**B.** Offer the customer a discount code first, to defuse the frustration before attempting to resolve the actual issue.

**C.** Acknowledge the frustration and offer to resolve it now, and escalate if the customer then says they would prefer a human.

**D.** Escalate immediately, because negative sentiment is a reliable signal that a human agent is needed for the conversation.

**Question 10.** The team proposes escalating whenever the agent's self-reported confidence is below 7 out of 10. In a pilot, the agent rated itself 8 to 9 on several complex cases it got wrong. What does this show, and what is the better approach?

**A.** Self-reported confidence is poorly calibrated; use explicit escalation criteria with examples, validated on labelled cases.

**B.** Average the agent's confidence across three repeated runs of the same case, and escalate whenever the average falls below 7.

**C.** The threshold is too low. Raise it to 9 out of 10, so that only the highest-confidence cases are handled without escalation.

**D.** The agent needs more context. Add the full policy manual to the system prompt so its confidence ratings become accurate.

# Scenario 2: Code Generation with Claude Code

**Scenario context.** Your engineering team uses Claude Code for daily feature development, refactoring, and debugging in a TypeScript monorepo with packages/api and packages/web. You are responsible for configuring Claude Code for the team, including CLAUDE.md, custom commands, skills, and working practices.

**Question 11.** packages/api and packages/web follow different conventions (error handling, file naming, testing style). Developers usually work inside one package at a time. What is a simple, effective way to give Claude package-specific guidance?

- **A.** Add a CLAUDE.md inside each package directory with that package's conventions, alongside the shared root CLAUDE.md.
- **B.** Create two separate user-level CLAUDE.md files and have each developer switch between them depending on the package.
- **C.** Ask developers to paste the relevant package's conventions into the first message of every Claude Code session they start.
- **D.** Put both packages' conventions in the root CLAUDE.md, and rely on Claude to work out which section applies to the current file.

**Question 12.** The team frequently scaffolds new API endpoints with a standard structure, and wants a shared /new-endpoint command that takes the endpoint name as an argument. How should it be implemented?

- **A.** A shell alias that each developer adds to their profile, which launches Claude Code with a long, prefilled scaffolding prompt.
- **B.** A Markdown file in each developer's ~/.claude/commands/ directory, with setup instructions documented in the project README.
- **C.** A Markdown file in the project's .claude/commands/ directory that uses $ARGUMENTS for the endpoint name, committed to the repo.
- **D.** A section in CLAUDE.md that explains the scaffolding steps, which Claude follows whenever someone mentions a new endpoint.

**Question 13.** A 400-line database migration procedure in CLAUDE.md is needed about once a month, but it is loaded into every session. What is the better home for it?

- **A.** A separate file imported into CLAUDE.md with @import, so the procedure is maintained apart from the other conventions.
- **B.** A user-level CLAUDE.md for the one engineer who usually runs migrations, so it no longer affects anyone else's sessions.
- **C.** A shorter summary in CLAUDE.md, with the details dropped, so the always-loaded memory stays within a reasonable size.
- **D.** A skill, so the procedure loads on demand when a migration task needs it, rather than in every session.

**Question 14.** The team must migrate from one ORM to another across about 30 files. The approach (adapter layer or direct rewrite) is not yet decided. What is the most effective Claude Code workflow?

- **A.** Use direct execution immediately with the direct-rewrite approach, since it is the most common migration strategy in practice.
- **B.** Use plan mode for the whole migration, including the implementation, so every edit happens within the planning context.
- **C.** Use plan mode to investigate usage and compare approaches, agree on a plan, then switch to direct execution to implement it.
- **D.** Ask Claude to implement both approaches in full, on separate branches, and then compare the resulting diffs side by side.

**Question 15.** Claude Code's implementations of a pricing function keep failing edge cases (zero quantities, currency rounding, bulk tiers) that the developer explains in prose after each attempt. Which iteration technique is most effective?

**A.** Write a test suite first that covers the expected behaviour and edge cases, then iterate by sharing the test failures with Claude.

**B.** Explain all the edge cases in one long, detailed prose message, and ask Claude to rewrite the function from scratch to handle them.

**C.** Ask Claude to add extensive logging to the function, then review the logs manually after each attempt to find what went wrong.

**D.** Ask Claude to write the function in a more defensive style, adding input validation throughout as a general safety measure.

**Question 16.** A developer finds three bugs in the same checkout function. Fixing one changes the behaviour the others depend on. How should the developer present them to Claude Code?

**A.** Describe them one at a time in separate messages, waiting for each fix before mentioning the next bug.

**B.** Describe all three in a single message, because the fixes interact and need to be designed together.

**C.** Open three separate sessions, one per bug, so that each fix is developed independently in a clean context.

**D.** Describe only the most severe bug, since the other two will probably disappear once that one is fixed.

**Question 17.** After a shared analysis of the test suite, a developer wants to compare a "mock everything" strategy with an "integration-first" strategy, keeping both explorations clean. What should they do?

**A.** Fork the session after the shared analysis, and pursue each strategy in its own branch.

**B.** Ask Claude to simulate both strategies mentally and report which would work better, without writing code.

**C.** Start two fresh sessions without the analysis, so neither strategy is biased by earlier conclusions.

**D.** Pursue both strategies in the same session, labelling each attempt clearly in the conversation.

**Question 18.** A developer resumes yesterday's named Claude Code session with --resume. Overnight, teammates merged changes to four of the files that the session analysed. What is the best way to continue?

**A.** Always discard the session and start from scratch, since any change to analysed files invalidates all prior context.

**B.** Resume and tell Claude which files changed, so it can re-read them and update its analysis before continuing.

**C.** Resume and continue as normal, since Claude Code automatically detects and reloads any files that changed on disk.

**D.** Resume, and ask Claude to repeat its entire analysis of the whole codebase from the beginning before doing anything else.

**Question 19.** A developer asks Claude Code to find every place in the codebase where the function calculateTax is called. Which built-in tool is the right first choice?

**A.** Bash running find with a name filter, so the search covers hidden directories too.

**B.** Glob, because it finds files by name pattern such as **/*tax*.ts across the repository.

**C.** Read, applied to every TypeScript file in turn, so that no call site can be missed.

**D.** Grep, because it searches file contents for a pattern such as calls to calculateTax.

**Question 20.** Claude Code's Edit operation fails because the anchor text it targets appears in several places in the file. What is the appropriate fallback?

**A.** Retry the same Edit several times, since the failure is intermittent and will eventually apply to the first match.

**B.** Use Read to load the file and Write to save the full modified contents, when no unique anchor is available.

**C.** Delete the duplicate occurrences of the anchor text first, so that the original Edit can then succeed.

**D.** Give up on the change and ask the developer to apply it by hand in their editor.

# Scenario 3: Multi-Agent Research System

**Scenario context.** You are designing a research system with the Claude Agent SDK. A coordinator agent delegates to specialised subagents: web search, document analysis, synthesis, and report generation. The system produces cited research reports on complex business questions.

**Question 21.** Reviewers say reports sometimes have thin coverage on parts of the question, even though every subagent completed its task. Which coordinator behaviour best addresses this?

- **A.** Have the report-generation agent fill thin sections from the model's general knowledge when sources are sparse.
- **B.** Increase every subagent's max_tokens, so that each one returns longer and more detailed findings to the coordinator.
- **C.** Run each subagent twice with identical instructions, and merge both outputs to increase the volume of material collected.
- **D.** An iterative loop: the coordinator checks the synthesis for gaps and re-delegates targeted research until coverage is adequate.

**Question 22.** The coordinator currently gives subagents step-by-step procedures ("search X, then open the top 3 results, then extract Y"). Results are poor when sources do not fit the expected pattern. How should subagent prompts be written?

- **A.** State the research goal, scope, and quality criteria, and let the subagent decide how to achieve them.
- **B.** Remove all instructions except the topic, so that subagents have complete freedom in how they work.
- **C.** Give every subagent the coordinator's full system prompt, so they understand the overall research intent.
- **D.** Make the procedures even more detailed, adding contingency steps for each type of source pattern seen.

**Question 23.** When passing document-analysis results to the synthesis agent, the coordinator currently pastes them as free text. Source URLs and page numbers are often lost. What is the best format for this hand-off?

- **A.** A link to the analysis agent's full transcript, so the synthesis agent can look up any source details it needs.
- **B.** One concatenated block of prose that combines all findings, with sources mentioned inline wherever the analyst noted them.
- **C.** Structured data that separates content from metadata, with each finding carrying its source URL, document name, and page number.
- **D.** Only the final conclusions of the analysis, with sources omitted to keep the synthesis agent's context small.

**Question 24.** Two web-search subagents frequently return overlapping sources and findings, which wastes tokens and time. What is the best fix in the coordinator?

- **A.** Partition the scope explicitly, assigning each subagent distinct subtopics or source types, so their work does not overlap.
- **B.** Let the subagents share a live list of the URLs they have visited, so each one can skip sources the other has already read.
- **C.** Deduplicate overlapping findings in the synthesis step, so the final report does not repeat the same evidence.
- **D.** Run the subagents sequentially, so the second one can see the first one's full results before it begins searching.

**Question 25.** Some questions are simple factual lookups ("What was Company X's 2025 revenue?"), but the system always runs all four subagents. What design improvement is best?

**A.** Keep the full pipeline for consistency, but reduce each subagent's max_tokens for queries that look simple.

**B.** Route simple questions to a separate single-prompt system with no tools, which answers from model knowledge.

**C.** Have the coordinator choose which subagents to invoke based on query complexity, instead of always running all four.

**D.** Add a cache of previous reports, so repeated simple questions are answered from earlier full-pipeline runs.

**Question 26.** The document-analysis subagent hits a temporary rate limit on a source API. What is the best error-handling design?

**A.** Abort the subagent's entire task and return an error, so the coordinator can restart the analysis from scratch.

**B.** Mark the source as unavailable and continue silently, so the analysis proceeds with whatever sources remain.

**C.** Retry locally with backoff; escalate to the coordinator only if unrecoverable, including any partial results.

**D.** Immediately report the failure to the coordinator, which decides whether the subagent should retry the same request.

**Question 27.** The synthesis agent reported that two sources "contradict each other" on adoption rates (23% versus 41%). Investigation showed one figure was from 2019 and the other from 2024. What design prevents this misinterpretation?

**A.** Require subagents to include publication or data-collection dates in their structured outputs, so temporal differences are visible.

**B.** Exclude any source older than three years from the research, so that all figures come from a comparable period.

**C.** Have the synthesis agent average conflicting figures, and note that the sources showed some variation.

**D.** Instruct the synthesis agent to always prefer the higher figure when sources disagree, since adoption usually grows over time.

**Question 28.** Reports present financial comparisons as long paragraphs and news developments as tables, and readers find both hard to use. What should the report-generation design do?

**A.** Render each content type suitably: financial data as tables, news as prose, technical findings as lists.

**B.** Put all findings in prose paragraphs, since narrative is the most natural way to present research.

**C.** Use one uniform format for all content, so that every report looks consistent regardless of what it contains.

**D.** Put all findings in tables, since tables are always the most scannable format for business readers.

**Question 29.** The synthesis agent often needs to check simple facts (a date or a figure). Sending each check back through the coordinator adds about 40% latency. Complex research must still go through the coordinator. What is the best tool design?

**A.** Remove verification from synthesis entirely, and trust the findings exactly as the other subagents delivered them.

**B.** Give the synthesis agent a narrowly scoped verify_fact tool for simple lookups, and keep complex research routed through the coordinator.

**C.** Batch all verification requests until synthesis finishes, then send them to the coordinator together in one round trip.

**D.** Give the synthesis agent the full web-search and document-fetch toolset, so it can do whatever verification it needs on its own.

**Question 30.** The document-fetch tool returns raw HTML pages of 20,000 to 30,000 tokens, mostly navigation and scripts. Analysis is slow and costly. What is the best tool improvement?

**A.** Keep the raw HTML, but tell the analysis agent in its prompt to ignore navigation, scripts, and other boilerplate.

**B.** Increase the analysis agent's context window, so it can handle several full raw pages at the same time.

**C.** Truncate every page to its first 2,000 tokens, so that the tool's responses stay within a predictable budget.

**D.** Have the tool extract and return the main content plus key metadata (title, author, date, URL), instead of the raw HTML.

# Scenario 4: Developer Productivity with Claude

**Scenario context.** You are building an internal agent with the Claude Agent SDK that helps engineers understand unfamiliar codebases, generate boilerplate, and automate repetitive tasks. It uses the built-in tools (Read, Write, Edit, Bash, Grep, Glob) and connects to internal systems through MCP servers.

**Question 31.** An engineer asks the agent to list all Playwright spec files in the repository so they can be reorganised. Which built-in tool fits best?

- **A.** Glob, with a pattern such as **/*.spec.ts, because the task is finding files by path pattern.
- **B.** Grep, searching file contents for the word "test", since every spec file contains test blocks.
- **C.** Edit, with a no-op change on each candidate file, to confirm that it exists before listing it.
- **D.** Read, applied to each directory in turn, collecting any file whose name contains "spec".

**Question 32.** Asked "How does authentication work in this service?", the agent reads 60 files up front and runs out of useful context. Which exploration strategy is recommended?

- **A.** Use Glob to list all files, then Read them in alphabetical order until the context window is about 80% full.
- **B.** Ask the engineer to list the relevant files, since the agent cannot reliably find authentication code by itself.
- **C.** Read every file in the repository's src/ directory first, so the agent has complete knowledge before it starts answering.
- **D.** Explore incrementally: Grep for entry points such as auth middleware, then Read only the files they lead to.

**Question 33.** The team plans to spend two sprints building a custom MCP server for Jira. A widely used, maintained community Jira MCP server already covers their needs. What should the architect recommend?

- **A.** Build the custom server anyway, since owning the code is always safer than depending on community-maintained software.
- **B.** Use the existing community server for this standard integration, and reserve custom servers for team-specific workflows.
- **C.** Skip MCP entirely and have the agent call the Jira REST API through Bash with curl commands.
- **D.** Fork the community server immediately and rewrite it in the team's preferred language before using it.

**Question 34.** The internal code-search MCP server offers semantic search across all repositories, but the agent keeps using built-in Grep on the local checkout and misses results in other repos. What is the most effective fix?

- **A.** Remove the Grep tool from the agent, so the MCP code-search tool is the only search option available to it.
- **B.** Rename the MCP tool to "grep", so the agent's existing preference for Grep routes searches to the MCP server.
- **C.** Improve the MCP tool's description to say what it searches (all repos, semantically) and when to prefer it over Grep.
- **D.** Add a system prompt instruction to always use MCP tools before any built-in tools, whatever the task.

**Question 35.** Infrastructure conventions for Terraform files should apply only when the agent edits *.tf files. The team does not want them taking up context during application-code work. What is the best configuration?

- **A.** A section at the end of the root CLAUDE.md, marked as applying only to Terraform files, so Claude can skip it when irrelevant.
- **B.** A skill that engineers must remember to invoke manually before they ask the agent to edit any Terraform files.
- **C.** A separate CLAUDE.md in the user-level directory of each infrastructure engineer on the team.
- **D.** A rule file in .claude/rules/ with a paths glob such as **/*.tf, so the conventions load only when matching files are edited.

**Question 36.** A team skill, /gen-service, requires a service name and a port. Engineers often invoke it without them, and the skill then guesses. Which SKILL.md feature helps most?

- **A.** argument-hint in the frontmatter, so engineers are shown the expected parameters when they invoke the skill.
- **B.** allowed-tools in the frontmatter, so the skill cannot write any files until the parameters have been provided.
- **C.** context: fork in the frontmatter, so the skill runs in an isolated context where it cannot guess missing values.
- **D.** A longer skill description, so Claude invokes the skill automatically only when both values appear in the conversation.

**Question 37.** One engineer wants a variation of the shared /gen-service skill that adds their preferred logging library, without changing the team's version. What is the recommended approach?

- **A.** Ask the team to adopt the preferred logging library, so the shared skill can be updated for everyone at once.
- **B.** Edit the shared skill in the repository, and add a condition that checks the engineer's username at runtime.
- **C.** Copy the skill into the project's .claude/skills/ directory with the same name, so the newer version overrides the old one.
- **D.** Create a personal variant under a different name in their user-level skills directory (~/.claude/skills/).

**Question 38.** An engineer asks the agent to explain how a request flows through five services. The main agent's context is filling up with search output. What orchestration approach is best?

- **A.** Have the main agent investigate all five services sequentially in its own context, and summarise its findings at the very end.
- **B.** Delegate focused questions (for example, "trace the refund path in service B") to subagents, and have the main agent combine their summaries.
- **C.** Ask the engineer to split the question into five separate conversations, one for each service in the request path.
- **D.** Increase the main agent's max_tokens so its answer can cover all five services in full detail.

**Question 39.** The agent has Bash access. Security requires that it never run destructive commands such as rm -rf or git push --force, with no exceptions. What is the most reliable mechanism?

- **A.** A system prompt instruction that lists the forbidden commands and explains why they are dangerous in this environment.
- **B.** A PostToolUse hook that detects destructive commands after they run and alerts the security team to investigate.
- **C.** A PreToolUse hook that inspects Bash commands and blocks any that match the destructive patterns before they execute.
- **D.** Few-shot examples of the agent refusing destructive commands, included at the start of every session's context.

**Question 40.** A multi-phase task (understand the codebase, propose a design, generate code) runs in one long session, and by the third phase the agent has lost important details from the first. What is the best practice?

- **A.** Repeat the first phase's exploration at the start of each later phase, so its details are fresh in context.
- **B.** Run all three phases in a single prompt, so the model sees the entire task and all its information at once.
- **C.** Increase the temperature in later phases, so the agent explores more of its earlier context when reasoning.
- **D.** Summarise key findings at each phase boundary and pass the summaries into the next phase's starting context.

# Scenario 5: Claude Code for Continuous Integration

**Scenario context.** You are integrating Claude Code into a CI/CD pipeline. On each pull request it performs automated code review and generates tests, and findings are posted as PR comments. Developers have complained in the past about noisy, unhelpful automated feedback.

**Question 41.** The CI job that runs Claude Code hangs until it times out, apparently waiting for interactive input. What is the fix?

- **A.** Run Claude Code inside a virtual terminal emulator in the CI container, so the interactive session can complete.
- **B.** Run Claude Code with the -p (print) flag, so it executes non-interactively and exits after producing its output.
- **C.** Increase the CI job's timeout to 60 minutes, so Claude Code has enough time to finish the review on its own.
- **D.** Pipe the yes command into Claude Code, so any interactive confirmation prompts are answered automatically.

**Question 42.** Generated tests ignore the team's conventions (shared fixtures, naming, the factories in test/utils). How should this context be provided to Claude Code in CI?

- **A.** Post the conventions as a PR comment before each run, so Claude Code reads them from the pull request.
- **B.** Rely on Claude Code to infer conventions by reading a random sample of existing tests at the start of each CI run.
- **C.** Document testing standards, fixtures, and conventions in the repository's CLAUDE.md, which Claude Code loads in CI as well.
- **D.** Hard-code the conventions into the CI script's prompt string, and update the script whenever the conventions change.

**Question 43.** When developers push fixes, the review re-runs and re-posts many of the same comments, including issues that are already fixed. How should the re-review be designed?

- **A.** Post comments only for files changed in the latest push, whatever was reported on earlier pushes.
- **B.** Include the prior review findings in context, and instruct Claude to report only new issues or ones that are still unaddressed.
- **C.** Run the review only once per pull request, on the first push, and skip any later pushes to that branch.
- **D.** Delete all earlier review comments before each run, so that only the latest set of findings is visible on the PR.

**Question 44.** The reviewer labels similar issues inconsistently: a missing null check is "critical" in one PR and "minor" in another. What improves severity consistency most?

- **A.** Reduce the scale to two levels (important and unimportant), so there is less room for inconsistency.
- **B.** Ask the reviewer to explain its reasoning before assigning each severity, without any further guidance.
- **C.** Average the severity across three independent review runs and post the rounded result.
- **D.** Define each severity level with explicit criteria and concrete code examples of issues that belong at that level.

**Question 45.** Generated tests often duplicate scenarios that the existing test suite already covers. What is the most effective fix?

- **A.** Generate fewer tests per run, so any duplication has a smaller impact on the suite.
- **B.** Include the existing relevant test files in context, so Claude can see what is covered and target the gaps.
- **C.** Instruct Claude to generate only unusual edge-case tests, since common paths are probably already tested.
- **D.** Deduplicate tests after generation by comparing test names against the existing suite.

**Question 46.** The nightly test-generation job uses the Message Batches API. Results must be available within 30 hours of a commit. Batches can take up to 24 hours. Which submission schedule meets the requirement?

**A.** Submit a batch every 12 hours, which halves the number of batch jobs compared with a 6-hour schedule.
**B.** Submit every 6 hours: worst case is 6 hours waiting plus 24 hours of processing, which is 30 hours.
**C.** Submit one batch per day at midnight, since most batches finish well within 24 hours in practice.
**D.** Submit each commit synchronously instead, because the Batches API cannot meet any time-bound requirement.

**Question 47.** The team wants human reviewers to focus their attention on the review findings most likely to be wrong. Which output design supports this?

**A.** Report a confidence level with each finding, and route low-confidence findings to human verification.
**B.** Post only high-severity findings, on the assumption that they are the ones most likely to be accurate.
**C.** Ask the reviewer to omit any finding it is unsure about, so every posted finding can be trusted without review.
**D.** Have humans check a random 10% of all findings each week, regardless of the type or content of each finding.

**Question 48.** A script posts findings as inline PR comments, but the free-text output is inconsistent (sometimes no line numbers, sometimes several issues merged into one paragraph). What prompt and output design is best?

**A.** Ask for a Markdown table of findings, which the script can convert into individual comments by parsing the table rows.
**B.** Ask for a numbered list of findings, and parse each line with regular expressions to recover file and line references.
**C.** Keep free text, and have a second model call rewrite the findings into the format the posting script expects.
**D.** Specify a structured format per finding (file, line, severity, issue, suggested fix) and enforce it through a JSON schema.

**Question 49.** The reviewer is told to "check that comments are accurate". It flags dozens of harmless comments for style or wording, and developers ignore the category. How should the instruction be rewritten?

**A.** Flag all comments that could be clearer, but label those findings as low severity.
**B.** Flag a comment only when it is longer than three lines and has not been updated recently.
**C.** Flag a comment only when the behaviour it describes contradicts what the code actually does.
**D.** Remove the comment-accuracy check entirely, since it is too subjective to automate.

**Question 50.** The CI review must cover three independent aspects: security, performance, and test adequacy. A single combined pass gives shallow results on each. What design is best?

**A.** Run focused, independent passes for each aspect (in parallel where possible), then aggregate the findings into one review.
**B.** Run one combined pass with a larger max_tokens budget, so the reviewer can write more about each of the three aspects.
**C.** Review only security in CI, and leave performance and test adequacy to human reviewers.
**D.** Rotate the aspects across pull requests, reviewing security on one PR and performance on the next.

# Scenario 6: Structured Data Extraction

**Scenario context.** You are building a system that extracts structured data from supplier documents (invoices, receipts, and contracts) using tool use with JSON schemas. Outputs feed a finance system downstream. The system must handle varied layouts, validate results, and process large volumes cost-effectively.

**Question 51.** Extracted dates arrive as "03/04/2026", "4 March 2026", and "2026-03-04", and amounts appear with or without currency symbols. The schema types are all string. What is the best way to get consistent values?

**A.** Extract each value twice in different formats, and let downstream code pick whichever version parses successfully.
**B.** State normalisation rules (ISO-8601 dates, ISO 4217 currency codes) in the prompt and schema descriptions.
**C.** Leave the values as extracted, and let the finance system's import routine work out each format on its own.
**D.** Add a few-shot example that uses ISO dates, and rely on Claude to infer the currency format from context.

**Question 52.** Supplier invoices vary widely in layout: some have line-item tables, some list items in prose, and some split totals across pages. Extraction quality varies sharply by layout. Which prompt technique helps most?

**A.** A single example of a perfectly formatted invoice, which Claude can use as the template for all the others.
**B.** A longer prose description of every possible invoice layout, with rules for where to find each field in each one.
**C.** An instruction to extract only from the first page, where the most important invoice information usually appears.
**D.** Few-shot examples showing correct extraction from each major layout type, so Claude learns how to handle the structural variety.

**Question 53.** In a 10,000-document batch, 240 requests failed. Some exceeded context limits and others had transient errors. What is the correct recovery approach?

**A.** Process the failed documents manually, since documents that fail once are likely to fail again.
**B.** Resubmit the entire 10,000-document batch, so that all the results come from one consistent processing run.
**C.** Use each failed request's custom_id to resubmit only those documents, chunking the oversized ones before resubmitting.
**D.** Resubmit the failed documents unchanged in a new batch, since batch failures are transient and usually succeed on retry.

**Question 54.** Before running a 50,000-document extraction batch, what is the most cost-effective preparation?

**A.** Split the volume into 50 batches of 1,000, and adjust the prompt slightly between each batch submission.
**B.** Submit the full batch immediately, then fix the prompt based on whatever failures appear in the results.
**C.** Run the full volume synchronously first to measure quality, and switch to batch processing for later runs.
**D.** Refine the prompt and schema on a representative sample to maximise first-pass success before the full run.

**Question 55.** Some supplier contracts contain internally inconsistent data (for example, the term is "12 months" in one clause and "24 months" in another). The extraction currently picks one value silently. How should the schema handle this?

**A.** Instruct Claude to choose the value that appears most often, and record the term field as if no conflict had been found.
**B.** Instruct Claude to always choose the value that appears first in the document, so the selection is at least deterministic.
**C.** Leave the term field empty whenever conflicting values appear, so no possibly wrong value is passed downstream.
**D.** Add a conflict_detected flag with a field for the conflicting values and their locations, so the inconsistencies are surfaced for review.

**Question 56.** Claude sometimes places the delivery date in invoice_date, or the supplier's tax ID in the customer_tax_id field. What schema change helps most?

**A.** Add precise field descriptions in the schema that explain each field's meaning and how to tell it apart from similar fields.

**B.** Rename the fields to shorter names, so that there is less text for Claude to misread when filling in the schema.

**C.** Reorder the fields alphabetically, so Claude processes them in a predictable sequence for every document.

**D.** Make every date field and ID field optional, so that Claude can skip any field it finds confusing.

**Question 57.** Leadership wants to route only uncertain extractions to human review. What is the best way to set this up?

**A.** Ask Claude for one overall confidence score per document, and send to review every document scoring below 50.

**B.** Output field-level confidence, calibrate thresholds on a labelled validation set, and route low scores to review.

**C.** Send every document to review for the first year, and then remove review entirely once the team trusts the system.

**D.** Send documents to review based on file size, since longer documents are the most likely to contain extraction errors.

**Question 58.** High-confidence extractions now bypass human review. How can the team detect new error patterns that might appear in those extractions over time?

**A.** Review high-confidence extractions only when the overall pipeline error rate exceeds a fixed alert threshold.

**B.** Run ongoing stratified random sampling of high-confidence extractions for human review, and track error rates by segment.

**C.** Rely on downstream finance users to report any errors they happen to notice in the course of their normal work.

**D.** Re-run each high-confidence extraction a second time, and flag the document only if the two outputs differ.

**Question 59.** Every document goes through the same steps: classify the type, extract with the matching schema, validate, then export. Which orchestration approach is most appropriate?

**A.** A single prompt that classifies, extracts, validates, and exports in one call, to reduce the number of API requests.

**B.** A coordinator with four subagents running in parallel, one for each step, to minimise per-document latency.

**C.** A fixed sequential pipeline (prompt chaining) with validation between steps, since the steps never change.

**D.** An autonomous agent with all the tools, which decides for each document which steps to perform and in what order.

**Question 60.** Some cases involve a contract plus three amendments, and the final terms depend on reading them together. How should extraction be orchestrated?

**A.** Extract only the most recent amendment, since it supersedes everything that came before it in the contract chain.

**B.** Extract each document separately, and pass all four outputs downstream for the finance team to reconcile.

**C.** Extract each document separately, then run a reconciliation pass that applies the amendments in order.

**D.** Concatenate all four documents and extract once, trusting Claude to resolve every amendment in a single call.

# Answers & Explanations

Each answer is tagged with the exam domain it tests.

## Scenario 1: Customer Support Resolution Agent

**Question 1** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: C**

Multi-concern requests should be decomposed, investigated with shared context, and answered together. A and B push work back onto the customer. D over-escalates cases the agent can handle.

**Question 2** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: B**

A single response can contain both text and tool_use blocks. stop_reason is the reliable control signal. C fights normal model behaviour. A is a fragile heuristic. D discards useful content and still mishandles the control flow.

**Question 3** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: A**

Policy gaps and exceptions are standard escalation triggers. C and D have the agent make up policy in opposite directions. B invents both a process and an authority the agent does not have.

**Question 4** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: D**

When multiple records match, ask for clarifying identifiers rather than choosing heuristically. C risks acting on the wrong account. A exposes other customers' data. B over-escalates a situation that is easy to resolve.

**Question 5** · Domain 2: Tool Design & MCP Integration

**Correct answer: A**

Classifying the error as transient and retryable drives the right recovery. C is a silent failure that looks like data. D misclassifies the error. B is not actionable for the agent.

**Question 6** · Domain 2: Tool Design & MCP Integration

**Correct answer: C**

Agents perform best with a small, role-relevant tool set. Irrelevant tools increase misuse and selection errors. D relies on an invalid ordering assumption. A ignores the cost of tool overload. B keeps the capability but blocks it only by instruction.

**Question 7** · Domain 2: Tool Design & MCP Integration

**Correct answer: B**

MCP resources give agents visibility of available content and remove trial-and-error discovery. A bloats every request. C accepts the waste. D speeds up the calls but does not remove them.

**Question 8** · Domain 5: Context Management & Reliability

**Correct answer: D**

Critical transactional details should live in a persistent structured block, not depend on lossy summaries. B still risks loss. A breaks context limits on long cases. C degrades the experience.

**Question 9** · Domain 5: Context Management & Reliability

**Correct answer: C**

Frustration on its own does not mean the case is complex. Acknowledge it, resolve the issue, and respect any explicit request for a human. D uses sentiment as a poor proxy. A damages the experience. B invents compensation the policy may not allow.

**Question 10** · Domain 5: Context Management & Reliability

**Correct answer: A**

Models can be confidently wrong, so self-ratings are not a reliable escalation signal. B and C adjust an uncalibrated signal. D is a guess that does not address calibration.

## Scenario 2: Code Generation with Claude Code

**Question 11** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: A**

Directory-level CLAUDE.md files scope guidance to the part of the codebase being worked on. D relies on inference and loads irrelevant rules. C is manual and inconsistent. B is personal, not shared, and error-prone.

**Question 12** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: C**

Project commands with argument substitution are shared through version control and invoked consistently. B is personal. D is not an explicit, parameterised command. A lives outside Claude Code's command system and drifts.

**Question 13** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: D**

CLAUDE.md is for always-relevant standards, and skills are for task-specific workflows used on demand. C loses needed detail. A still loads it into every session. B makes team knowledge depend on one person.

**Question 14** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: C**

Plan mode suits investigation and design when several approaches are viable, and direct execution suits implementing an agreed plan. A commits blindly. D wastes effort. B misuses plan mode, which is for exploration and design, not carrying out the edits.

**Question 15** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: A**

Test-driven iteration gives an objective, repeatable specification and precise feedback. B repeats the prose approach that failed. C diagnoses after the fact. D is vague and does not encode the expected behaviour.

**Question 16** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: B**

Interacting issues should be addressed together so the fixes are consistent. Independent issues can be handled one after another. A and C produce fixes that may conflict. D is a gamble.

**Question 17** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: A**

Forking gives two independent branches from the same baseline. D cross-contaminates the explorations. C throws away the shared analysis. B replaces evidence with speculation.

**Question 18** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: B**

The resumed context contains stale tool results. Pointing Claude to the changed files allows targeted re-analysis. (When much of the context is stale, a fresh session with a summary is better.) C assumes automatic refresh. A and D waste valid work.

**Question 19** · Domain 2: Tool Design & MCP Integration

**Correct answer: D**

Finding callers is a content search, which is Grep's job. A and B match file names, not usages. C is far too expensive and fills the context.

**Question 20** · Domain 2: Tool Design & MCP Integration

**Correct answer: B**

Edit needs a unique match. Read plus Write is the reliable fallback, or Claude can widen the anchor to make it unique. A will not change the outcome. C corrupts the file. D gives up unnecessarily.

## Scenario 3: Multi-Agent Research System

**Question 21** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: D**

Evaluating output against the question and re-delegating closes gaps systematically. B and C add volume, not targeted coverage. A presents uncited model knowledge as research.

**Question 22** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: A**

Goal- and criteria-based delegation lets capable subagents adapt to what they find. D remains brittle. B removes the guidance needed for quality. C bloats context with instructions meant for coordination.

**Question 23** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: C**

Structured hand-offs keep attribution metadata attached to each finding. B loses metadata unpredictably. D drops provenance. A is not available to an isolated subagent and wastes context.

**Question 24** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: A**

Clear scope partitioning prevents duplication up front. B creates coupling between subagents. D gives up parallelism. C removes duplicates only after the cost has been paid.

**Question 25** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: C**

Dynamic selection of subagents matches effort to the query. A still runs unnecessary agents. D helps only with exact repeats. B loses sourcing and verification.

**Question 26** · Domain 5: Context Management & Reliability

**Correct answer: C**

Transient errors should be handled locally where possible, and only unresolved failures escalate, with useful context. D pushes routine recovery onto the coordinator. B hides a gap. A discards work that succeeded.

**Question 27** · Domain 5: Context Management & Reliability

**Correct answer: A**

Dates as structured metadata let synthesis tell a trend from a conflict. D is an arbitrary rule. B can discard valuable historical data. C produces a meaningless number.

**Question 28** · Domain 5: Context Management & Reliability

**Correct answer: A**

Matching the format to the content type preserves meaning and readability. B, C, and D force one format on content it does not suit.

**Question 29** · Domain 2: Tool Design & MCP Integration

**Correct answer: B**

A scoped tool for the common simple case cuts latency while keeping the separation of roles. D reintroduces tool overload and scope creep. A lowers quality. C delays checks the synthesis needs while it is writing.

**Question 30** · Domain 2: Tool Design & MCP Integration

**Correct answer: D**

Tools should return high-signal, token-efficient content. A still pays for the noise. C may cut off the actual content. B accepts the waste.

## Scenario 4: Developer Productivity with Claude

**Question 31** · Domain 2: Tool Design & MCP Integration

**Correct answer: A**

Matching file paths is Glob's job. B searches contents, returns noisy matches, and misses the intent. D is not how directory listing works and wastes context. C misuses a write tool.

**Question 32** · Domain 2: Tool Design & MCP Integration

**Correct answer: D**

Targeted search followed by focused reading uses context efficiently. A and C flood the context. B gives up a core capability of the agent.

**Question 33** · Domain 2: Tool Design & MCP Integration

**Correct answer: B**

Prefer established servers for standard integrations, after a normal security review. A duplicates effort. C loses the tool abstraction and adds risk. D adds cost without benefit.

**Question 34** · Domain 2: Tool Design & MCP Integration

**Correct answer: C**

Agents choose tools based on their descriptions. A weak description loses to a familiar built-in. A removes a useful local tool. D is a blunt rule that misfires. B creates a confusing name collision.

**Question 35** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: D**

Path-scoped rules give conditional loading tied to file patterns. A always loads the text. B depends on human memory. C is personal and inconsistent.

**Question 36** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: A**

argument-hint shows the required inputs at invocation time. C isolates context but does not prompt for parameters. B restricts tools but not guessing. D affects automatic invocation, not manual use.

**Question 37** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: D**

Personal variants belong in user scope, under a distinct name. B pollutes shared configuration. C causes conflicting duplicate names. A turns a personal preference into a team decision.

**Question 38** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: B**

Subagents isolate verbose exploration and return condensed findings to the coordinating agent. A is the cause of the context overflow. C offloads the coordination to the human. D affects output length, not input context.

**Question 39** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: C**

Hooks that run before execution enforce guarantees in code. A and D are probabilistic. B reacts after the damage is done.

**Question 40** · Domain 5: Context Management & Reliability

**Correct answer: D**

Structured summaries at phase boundaries carry the important facts forward in compact form. B overloads a single call. A wastes work and context. C is irrelevant.

## Scenario 5: Claude Code for Continuous Integration

**Question 41** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: B**

-p is the documented non-interactive mode for automation. D is a hack that does not address the cause. C only waits longer. A keeps the interactive mode the pipeline cannot use.

**Question 42** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: C**

CLAUDE.md is the project-level memory mechanism and works the same in CI, so it is the single source of truth. D duplicates and drifts. B is unreliable. A is not a mechanism Claude Code uses for context.

**Question 43** · Domain 3: Claude Code Configuration & Workflows

**Correct answer: B**

Giving the reviewer its prior findings lets it suppress duplicates and recognise fixes. D loses history and still repeats issues. C misses new problems. A misses issues that span files or remain from earlier.

**Question 44** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: D**

Explicit definitions anchored in examples calibrate the judgement. A loses useful information. B adds reasoning without a standard to apply. C averages an uncalibrated signal.

**Question 45** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: B**

Claude cannot avoid duplicating coverage it cannot see. C is a guess that skips real gaps. D misses duplicates with different names. A reduces value without fixing the cause.

**Question 46** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: B**

Worst case equals the submission interval plus 24 hours of processing. A 6-hour interval gives 30 hours. C can reach 48 hours. A can reach 36 hours. D gives up the savings unnecessarily.

**Question 47** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: A**

Per-finding confidence lets human attention be routed to the right findings. Its calibration should be checked against labelled outcomes. D is not targeted. B conflates severity with accuracy. C silently drops uncertain but real issues.

**Question 48** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: D**

A defined schema per finding gives reliable, machine-usable output. A and B rely on brittle parsing of loosely formatted text. C adds cost and another failure point.

**Question 49** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: C**

A precise, behavioural criterion targets the real risk (misleading comments) and removes noise. B uses irrelevant proxies. A keeps the noise. D discards a useful check that a clear criterion makes reliable.

**Question 50** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: A**

Independent, focused passes give each aspect full attention, and aggregation keeps one coherent output. B does not fix divided attention. D leaves most PRs unreviewed for most aspects. C drops required coverage.

## Scenario 6: Structured Data Extraction

**Question 51** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: B**

A schema enforces structure, and normalisation rules define the content format. Stating both gives consistent values. C pushes ambiguity downstream. D is partial and implicit. A doubles the work and the ambiguity.

**Question 52** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: D**

Diverse examples covering the real structural variation teach Claude how to generalise. B is hard to apply. A overfits to one layout. C misses data on later pages.

**Question 53** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: C**

custom_id links results to inputs, so you can retry only the failures, with fixes appropriate to each cause. B wastes cost. D fails again on the oversized documents. A gives up on automation unnecessarily.

**Question 54** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: D**

Iterating on a sample is cheap, and failures at full scale are expensive to redo. B pays for avoidable failures. C gives up the batch discount. A gives inconsistent outputs and an uncontrolled experiment.

**Question 55** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: D**

Making conflicts explicit preserves the information and allows correct handling downstream. A and B hide the problem. C discards both values and the reason they were dropped.

**Question 56** · Domain 4: Prompt Engineering & Structured Output

**Correct answer: A**

Field descriptions guide the model when fields look similar. B removes useful meaning. C is irrelevant. D turns confusion into missing data instead of correct data.

**Question 57** · Domain 5: Context Management & Reliability

**Correct answer: B**

Field-level confidence, calibrated against labelled data, targets review effort where errors occur. A is too coarse and uncalibrated. D is a weak proxy. C wastes effort and then removes oversight without evidence.

**Question 58** · Domain 5: Context Management & Reliability

**Correct answer: B**

Continuous stratified sampling detects new failure modes even in the automated segment. C is passive and incomplete. D misses consistent errors. A cannot see errors in records that are never reviewed.

**Question 59** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: C**

Known, repeatable steps suit a fixed chain, which is predictable, testable, and cheap. D adds variance without benefit. B cannot parallelise steps that depend on each other. A removes the validation gate between steps.

**Question 60** · Domain 1: Agentic Architecture & Orchestration

**Correct answer: C**

Per-document passes followed by a dedicated reconciliation step keeps each extraction focused and handles the dependencies explicitly. D overloads one call. A is wrong, because amendments usually modify only some terms. B pushes the core task onto humans.

# Answer Key & Domain Score Sheet

| Q | Ans | Dom | Q | Ans | Dom | Q | Ans | Dom | Q | Ans | Dom |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | C | D1 | 16 | B | D3 | 31 | A | D2 | 46 | B | D4 |
| 2 | B | D1 | 17 | A | D1 | 32 | D | D2 | 47 | A | D4 |
| 3 | A | D1 | 18 | B | D1 | 33 | B | D2 | 48 | D | D4 |
| 4 | D | D1 | 19 | D | D2 | 34 | C | D2 | 49 | C | D4 |
| 5 | A | D2 | 20 | B | D2 | 35 | D | D3 | 50 | A | D1 |
| 6 | C | D2 | 21 | D | D1 | 36 | A | D3 | 51 | B | D4 |
| 7 | B | D2 | 22 | A | D1 | 37 | D | D3 | 52 | D | D4 |
| 8 | D | D5 | 23 | C | D1 | 38 | B | D1 | 53 | C | D4 |
| 9 | C | D5 | 24 | A | D1 | 39 | C | D1 | 54 | D | D4 |
| 10 | A | D5 | 25 | C | D1 | 40 | D | D5 | 55 | D | D4 |
| 11 | A | D3 | 26 | C | D5 | 41 | B | D3 | 56 | A | D4 |
| 12 | C | D3 | 27 | A | D5 | 42 | C | D3 | 57 | B | D5 |
| 13 | D | D3 | 28 | A | D5 | 43 | B | D3 | 58 | B | D5 |
| 14 | C | D3 | 29 | B | D2 | 44 | D | D4 | 59 | C | D1 |
| 15 | A | D3 | 30 | D | D2 | 45 | B | D4 | 60 | C | D1 |

| Domain | Weight | Your score | Question numbers |
|---|---|---|---|
| D1: Agentic Architecture & Orchestration | 27% | / 16 | 1, 2, 3, 4, 17, 18, 21, 22, 23, 24, 25, 38, 39, 50, 59, 60 |
| D2: Tool Design & MCP Integration | 18% | / 11 | 5, 6, 7, 19, 20, 29, 30, 31, 32, 33, 34 |
| D3: Claude Code Configuration & Workflows | 20% | / 12 | 11, 12, 13, 14, 15, 16, 35, 36, 37, 41, 42, 43 |
| D4: Prompt Engineering & Structured Output | 20% | / 12 | 44, 45, 46, 47, 48, 49, 51, 52, 53, 54, 55, 56 |
| D5: Context Management & Reliability | 15% | / 9 | 8, 9, 10, 26, 27, 28, 40, 57, 58 |
| **Total** | 100% | / 60 | |

**Reading your result:** 48+ overall with no domain below 70% suggests you are ready to book. For any weaker domain, re-read its explanations here, then test yourself on the same domain in Set 1 (the domain question bank).